

Iterable vs. Iterator in Java

Differences, usage, and how to implement each

1. Overview

Java provides two related but distinct interfaces for walking through the elements of a data structure: **Iterable** and **Iterator**. They are often confused because every **Iterable** ultimately hands out an **Iterator**, but they serve different roles and live in different packages.

2. The Iterable Interface

Iterable belongs to the **java.lang** package. It represents a data structure that *can be iterated over*. **Iterable** does not itself provide indexed access — when working with an **Iterable**, elements cannot be fetched by index, and the first or last element cannot be retrieved directly. Its single contract method produces an **Iterator**:

```
Iterator<E> iterator();
```

Every collection in the Java Collections Framework (**List**, **Set**, **Queue**, etc.) implements **Iterable**, which is precisely what allows them to be used in a **for-each** loop.

2.1 Iterating Over an Iterable

The enhanced **for** loop (**for-each** loop) only works on objects that implement **Iterable** — it is syntactic sugar that calls **iterator()** behind the scenes and drives it with **hasNext()/next()**. It can also be driven manually with a **while** loop and an explicit **Iterator**. Collections additionally expose a **forEach()** method that accepts a **lambda**, giving a third, more concise way to iterate:

```
for (Integer number : numbers) { ... }

numbers.forEach(System.out::println);
```

2.2 Implementing the Iterable Interface

Custom implementations of **Iterable** are useful for custom data structures that should be iterable without exposing their internal representation to callers. For example, a **ShoppingCart** class backed by an array should not force its clients to iterate over that array directly — if the backing structure ever changed, every caller would break. Instead, the class implements **Iterable** and overrides **iterator()**:

```
public class ShoppingCart<E> implements Iterable<E> {
    @Override
    public Iterator<E> iterator() {
        return new ShoppingCartIterator();
    }
}
```

A key detail: **each call to `iterator()` returns a brand-new `Iterator` instance**. This matters because an **Iterator** is responsible for holding the current iteration state (its cursor position). If **iterator()** returned the same shared instance every time, two independent **for-each** loops over the same collection would interfere

with each other's cursor position. A fresh instance per call keeps multiple simultaneous iterations independent.

Once `iterator()` is implemented, instances of the class can be used directly in a for-each loop, exactly like a built-in collection:

```
for (Product product : shoppingCart) {  
    System.out.println(product.getName());  
}
```

3. The Iterator Interface

Iterator is a member of the Java Collections Framework and belongs to the **java.util** package. Unlike `Iterable`, which only describes *that* something can be iterated, `Iterator` does the actual work of retrieving elements and, optionally, removing them during iteration.

Its methods are:

- **`hasNext()`** and **`next()`** — step through and retrieve elements one at a time.
- **`remove()`** — removes the current element pointed to by the `Iterator`.
- **`forEachRemaining(Consumer<? super E> action)`** — performs the given action for every element that has not yet been visited.

3.1 Iterating Over a Collection

Since `List` (and every other `Collection` subtype) extends `Iterable`, obtaining an `Iterator` is a matter of calling `iterator()` on it, then driving it with a while loop:

```
Iterator<Integer> iterator = numbers.iterator();  
  
while (iterator.hasNext()) {  
    System.out.println(iterator.next());  
}
```

`next()` returns the next element in the iteration; if no such element remains, it throws **`NoSuchElementException`** rather than returning null or silently failing.

3.2 Implementing the Iterator Interface

A custom `Iterator` implementation is useful whenever iteration needs conditional logic — for example, walking only the prime numbers, or only the odd-indexed elements, of an underlying collection. In such a case, `hasNext()` itself is responsible for scanning ahead to find (and remember) the position of the next qualifying element, not just checking whether any element exists:

```
public boolean hasNext() {  
    // scans forward from the cursor, advancing it to the  
    // next element that satisfies the custom condition  
}  
  
public Integer next() {  
    // returns the element hasNext() already located  
}
```

Concrete Iterator implementations are usually written as inner classes, since they are tightly coupled to one data structure and are rarely useful anywhere else. Each instance is responsible for maintaining its own iteration state (typically a cursor index or reference), and any `next()` implementation must throw `NoSuchElementException` once no elements remain — skipping this check can cause undefined or unexpected behavior for callers who rely on the contract.

4. Differences Between Iterable and Iterator

The following table summarizes the core distinctions:

Iterable	Iterator
Represents a collection that can be iterated over using a for-each loop	Represents the interface used to actually iterate over a collection
Implementing it requires overriding the <code>iterator()</code> method	Implementing it requires overriding the <code>hasNext()</code> and <code>next()</code> methods
Does not store the iteration state	Stores the iteration state (e.g. cursor position)
Removing elements during iteration isn't allowed	Removing elements during iteration is allowed, via <code>remove()</code>
Package: <code>java.lang</code>	Package: <code>java.util</code>

In short: an object is **Iterable** if it knows how to *produce* an iterator; an object is **an Iterator** if it knows how to actually *walk* the elements, one call to `next()` at a time, and remembers exactly where it is.

5. Summary

Aspect	Iterable	Iterator
Package	<code>java.lang</code>	<code>java.util</code>
Core method(s)	<code>iterator()</code>	<code>hasNext()</code> , <code>next()</code> , <code>remove()</code> , <code>forEachRemaining()</code>
Used by	for-each loop, <code>forEach()</code>	while loop driven manually, or internally by for-each
Holds cursor state?	No	Yes
Allows removal mid-iteration?	No	Yes, via <code>remove()</code>